EXPERIMENT 10

Performance Optimization in Zoho Creator

Name: Shrivarthini | Register Number: 241401100
Subject: Low Code Application Development using Zoho Creator

Aim

To demonstrate performance optimization techniques in the Vehicle Service & Repair Request Management application in Zoho Creator, particularly efficient record retrieval and workflow processing.

Introduction

Performance optimization in low-code applications means designing forms, workflows and scripts so that they use data and processing only as far as needed. Although a low-code platform handles much of the underlying work, the way in which a workflow is written still influences how efficiently it runs.

Retrieving more records than required, or repeatedly processing records that are not relevant to the task, causes unnecessary data retrieval and processing. In a small application this may not be noticeable, but as the number of service requests and users grows it can reduce application efficiency. A service center that receives many repair requests every day is a good example. Efficient workflows in Zoho Creator therefore retrieve only the data they need and avoid unnecessary processing, which supports scalable and maintainable applications.

Application Used

The practical was performed in Zoho Creator using the Vehicle Service & Repair Request Management application. Deluge scripting was used to write the logic of a form workflow that sends an email notification when a service request is submitted. No upgrade-only feature is claimed as enabled or configured in this record.

Performance Optimization Concept

The key optimization demonstrated is criteria-based record fetching. Instead of retrieving every record of a form, a criteria is given so that only the required record is returned. In the Service Request workflow, this is written as:

```
serviceRequest = Service_Request[ID == input.ID];
```

- The workflow identifies the current record using input.ID, which holds the ID of the service request being processed.
- Only the relevant Service Request record, the one whose ID matches input.ID, is retrieved.
- This avoids unnecessarily fetching and processing unrelated service requests.
- This improves workflow efficiency and reduces unnecessary data processing.

Procedure

1. Open the Vehicle Service & Repair Request Management application in Zoho Creator in Edit mode.
2. Navigate to the Workflow section and open Form workflows. The form Service Request is listed under Forms with workflows.
3. Observe the form workflow “Send service request confirmation”. It is triggered on Created → Successful form submission, has 1 action and has the status Enabled (Figure 1).

241401100

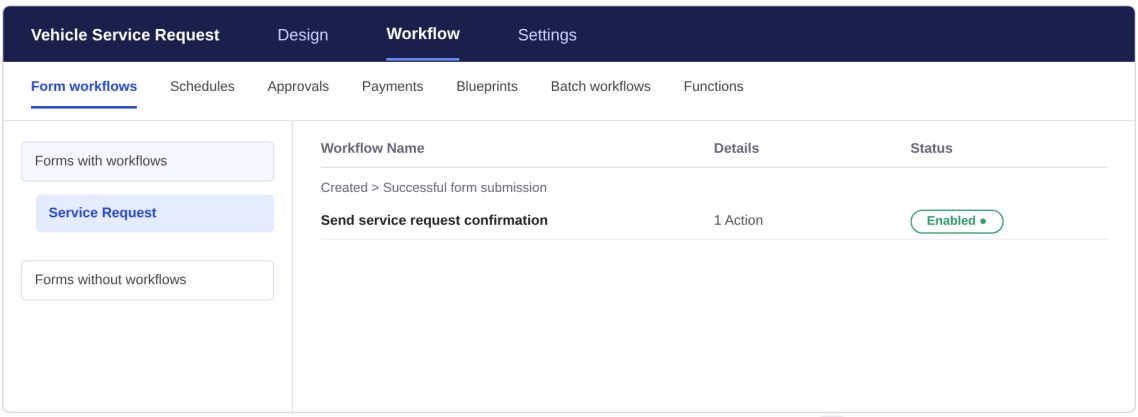


Figure 1: Form Workflow of the Vehicle Service & Repair Request Application (Illustrative Representation, Not a Screenshot)

4 When a Service Request is submitted, the workflow identifies the current record using input.ID.

5 The required record is fetched using the criteria ID == input.ID, so that only this record is retrieved.

6 The workflow checks whether the email field of the record is blank.

7 If the email is available, the confirmation email is sent to the customer. If the email is missing, an informational message is displayed.

The Deluge script used in the workflow is given in the next section.

2414011100

Deluge Code

The Deluge code of the workflow is as follows:

```
serviceRequest = Service_Request[ID == input.ID];

if(serviceRequest.email.isBlank())
{
    info "Email address is missing for this service request";
}
else
{
    sendmail
    [
        from :zoho.adminuserid
        to :serviceRequest.email
        subject :"Your Service Request is Confirmed"
        message :"<p>Dear Customer,</p><p>Your vehicle service request has been
received. Our service team will contact you shortly.</p><p>Thank you for
choosing us.</p>"
    ]
}
```

The first statement fetches the record by its ID. If the email field of that record is blank, an informational message is displayed. Otherwise, an email with the subject “Your Service Request is Confirmed” is sent to the customer’s email address.

Performance Analysis

The optimized approach is compared with a general inefficient approach below.

Approach	Steps
Before (inefficient approach)	Fetch all records → iterate through records → find the required record → process
Optimized (used in the workflow)	Fetch the required record using ID → process directly

The inefficient approach retrieves and processes records that are not needed, whereas the optimized approach reduces unnecessary record retrieval and processing. Figure 2 is a conceptual diagram that illustrates this comparison by showing how many records each approach retrieves. The “before” code given after it is an illustrative example only; it was not used in the application.

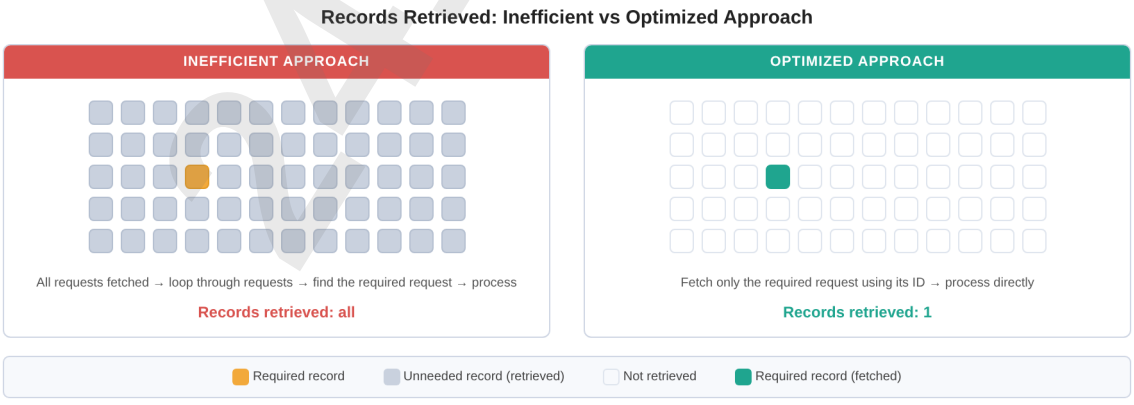


Figure 2: Records Retrieved by the Inefficient and Optimized Approaches (Conceptual Diagram)

Illustrative comparison only (not used in the application):

```
serviceRequests = Service_Request[];
for each rec in serviceRequests
{
    if(rec.ID == input.ID)
    {
        // ...
    }
}
```

No benchmark was performed in this practical, so no measured improvement in execution time is claimed. The comparison is based on the amount of data retrieved and processed.

Optimization Techniques

General practices that contribute to performance in Zoho Creator applications are given below and shown in Figure 3, a conceptual diagram.

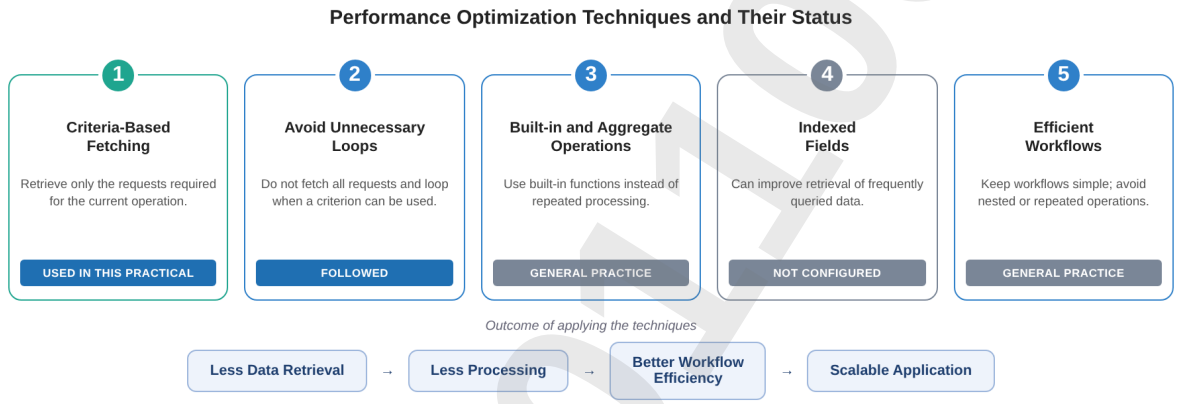


Figure 3: Performance Optimization Techniques (Conceptual Diagram)

Technique	Explanation
Criteria-based record fetching	Retrieve only the records required for the current operation. This technique is used in the practical.
Avoiding unnecessary record iteration	Do not fetch all records and loop through them when a specific criterion can be used.
Efficient built-in functions and aggregate operations	Use built-in functions and aggregate operations instead of unnecessary repeated processing.
Indexed fields	Indexed fields can improve retrieval performance for frequently queried data. Indexing may require backend / platform configuration, and it was not enabled or configured in this application.
Efficient workflows	Keep workflows simple and avoid unnecessary nested or repeated operations.

Optimized Workflow

Figure 4 is a conceptual flowchart of the optimized Service Request workflow:

Service Request Submitted → Identify Current Record → Criteria-Based Record Fetch → Validate Email → Prepare Notification → Send Email

24A0A100

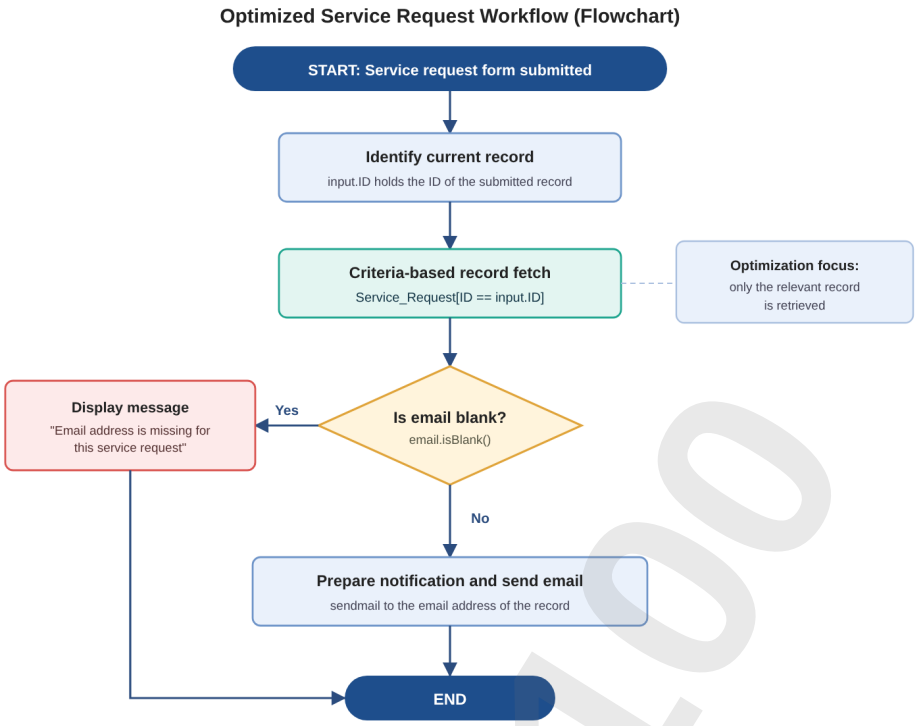


Figure 4: Optimized Service Request Workflow (Conceptual Diagram)

- **Identify current record:** input.ID gives the ID of the submitted record.
- **Criteria-based record fetch:** Service_Request[ID == input.ID] retrieves only the relevant record.
- **Validate email:** email.isBlank() checks whether the email field is empty.
- **Prepare notification and send email:** if the email is available, the confirmation message is prepared and sent using sendmail; otherwise an informational message is displayed.

Conclusion

Performance optimization is important in low-code applications because efficiency depends not only on the platform but also on how workflows and scripts are written. In this practical, the Service Request workflow used criteria-based record retrieval with the expression Service_Request[ID == input.ID], so that only the required record was retrieved and processed.

Targeted record retrieval improves efficiency by avoiding the retrieval of unrelated records and the iteration through them, and efficient workflow design reduces unnecessary processing. Practices such as criteria-based fetching, avoiding unnecessary loops, using built-in operations and keeping workflows simple help applications to handle growing amounts of data, such as the service requests of a busy garage. These practices contribute to scalable and maintainable Zoho Creator applications. No benchmark was performed, so the benefits are described in terms of reduced data retrieval and processing and not as measured execution times.

24401100